

이동형

DevOps Engineer | 경력기술서

핵심 역량 (Core Competencies)

클라우드 인프라 설계 및 운영

AWS, GCP 기반 멀티·하이브리드 클라우드 아키텍처 설계 및 운영 경험

- AWS EKS·GCP GKE 프로덕션 운영·비용 최적화 (하이브리드 전환 20% 절감, GCP 마이그레이션 50% 절감, 절감분 신규 프로젝트 재투자)
- Terraform IaC 인프라 자동화·버전 관리 (GCP 리소스 100% 코드화)

CI/CD 파이프라인 구축 및 GitOps

GitOps 기반 배포 자동화로 개발 생산성 극대화

- GitHub Actions·GitLab CI·ArgoCD 기반 CI/CD 파이프라인 설계·고도화
- 배포 시간 50~67% 단축, GitOps 지속 배포 전환으로 Time to Market(시장 대응 속도) 단축, 롤백 시간 95% 감소 (30분 → 1~2분)

모니터링 및 오피스빌리티 시스템 구축

장애 사전 감지 및 신속한 대응 체계 수립

- ELK → EFK Stack 전환 후 ECK Operator 기반 Elastic Stack 9.0 CRD 선언형 관리로 재설계 (TLS 자동 회전, 롤링 업그레이드 자동화)
- kube-prometheus-stack 기반 커스텀 알림 규칙·Grafana 대시보드·4종 DB exporter (MySQL, Redis, Elasticsearch, PostgreSQL) 통합

자동화 및 효율화

Python, Bash를 활용한 운영 자동화 및 개발 생산성 도구 구축

- 사내 LLM 서비스(Ollama + Open WebUI) 구축 — 사내 보안 가이드라인 100% 준수, 개발팀 생산성 향상
- DP사 운영팀이 Kubernetes를 스스로 운영하도록 정착 — 6개월 교육 프로그램 설계·운영 (수강자 평균 10명·회당 4시간), 교육 자료를 표준화해 타 고객사 교육에도 재활용

Kubernetes 플랫폼 운영 고도화 및 오픈소스 기여

네트워크·로깅 스택의 무중단 전환과 재사용 가능한 공용 차트 오픈소스 공개

- Ingress-nginx 다수 인스턴스 → NGINX Gateway Fabric 단일 컨트롤플레인 + 클래스별 Gateway CR 무중단 마이그레이션 (전체 cutover, wildcard TLS 통합)
- 오픈소스 Helm 차트 3종(nginx-gateway-cr, elasticsearch-eck, kibana-eck) 및 composite GitHub Actions 다수 공개, ArtifactHub / GitHub Marketplace 등록

데이터 엔지니어링 및 사내 LLM 서비스

GCP 데이터 파이프라인부터 사내 LLM 서비스까지 직접 구축·운영

- BigQuery + Dataflow + Cloud Functions + Cloud Scheduler 조합으로 멀티 데이터 소스(MongoDB·CloudSQL·GA·Dune) → BigQuery → Google Sheets 자동 파이프라인 구축, 인프라 100% Terraform IaC
- 온프레미스 GPU 서버 위 Ollama + Open WebUI 기반 사내 LLM 서비스 Kubernetes 배포·운영, 외부 AI SaaS 구독 대체로 월 약 \$100 절감 및 사내 보안 가이드라인 100% 준수

주요 경력 (Professional Experience)

팬텀(콘크리트 스튜디오)

2024.10 ~ 현재

DevOps Engineer

게임 개발사의 인프라를 단독으로 책임지는 DevOps 엔지니어로, AWS 기반 클라우드와 온프레미스 서버를 설계·구축·운영하고 있습니다. 개발 서버와 테스트 서버는 온프레미스에, QA와 Production 서버는 AWS에 구축하여 하이브리드 아키텍처를 운영 중이며, 이 구조로 월 인프라 비용을 20%(\$60,000 → \$48,000) 절감하고 절감분을 신규 프로젝트에 재투자하는 선순환을 만들었습니다.

현재 서비스 중인 라이브 게임에 대해 외부 퍼블리셔와 협업하여 2차 기술지원 및 운영을 담당하고 있으며, 개발 중인 신규 게임 프로젝트의 개발 생산성 향상을 위해 온프레미스·AWS 인프라를 구축하고 있습니다.

프로젝트 1: AWS-온프레미스 하이브리드 클라우드 아키텍처 구축

기여도 100% (단독 설계 및 구현, 외부 퍼블리셔 운영팀 협업) 2024.10 ~ 현재 (운영 중)

문제

전체 인프라를 AWS에서 운영하던 중 월 \$60,000의 높은 클라우드 비용이 발생하여 비용 최적화가 필요했습니다.

접근 전략

워크로드별 비용을 분석한 결과, 개발·테스트 환경은 트래픽 변동이 적어 클라우드의 자동 스케일링이 불필요했고, QA·Production 환경만 확장성과 안정성이 요구되는 상황이었습니다. 이에 개발·테스트 환경은 온프레미스로 전환하고, QA·Production 환경은 AWS에 유지하는 하이브리드 아키텍처를 설계했습니다.

양쪽 환경은 VPN 없이 역할을 분리해 독립 운영(앱 트래픽 분리)하며, 이미지 레지스트리로 환경별로 분리(개발=Harbor, QA·Production=ECR)해 아키텍처 차이에 맞춰 각 환경에서 독립적으로 빌드했습니다. 양쪽 환경 모두 Terraform과 Ansible을 활용한 IaC 통합으로 운영 표준을 일원화(신규 컴포넌트 IaC 100%, IAM 제외)하여, 운영 복잡도 증가 없이 일관된 인프라 관리 체계를 확립했습니다.

트러블슈팅

- EKS 환경에서 ALB Ingress 구성 시, 게임 클라이언트 버전별로 다른 백엔드로 라우팅해야 하는 요구사항 발생. ALB의 조건부 라우팅 규칙에서 커스텀 헤더 기반 라우팅과 Target Group을 조합하여 해결
- ArgoCD로 관리하는 프로덕션 서비스의 rolling deploy 중 ALB target deregistration delay(대상 등록 해제 대기)와 Pod SIGTERM 처리 시점이 어긋나 일부 세션 connection drop 발생.
3가지 조치로 connection drop 0건·zero-downtime 배포 달성 — (1) ALB drain 타이밍 정렬: preStop hook + readiness 우선 fail로 종료 중 신규 요청은 차단하되 in-flight 요청은 끝까지 처리, (2) RollingUpdate 전략을 replica 스케일에 맞춰 surge-first로 조정해 롤아웃 중 용량 유지, (3) 다중 replica 서비스에 PDB를 적용해 Karpenter consolidation(노드 통합·축소) 시 Pod 동시 축출 방지

성과

- 워크로드 분석 기반 인프라 최적화로 월 비용 20% 절감 (\$60,000 → \$48,000), 연간 약 \$144,000 절감분을 신규 프로젝트 인프라에 재투자하여 리소스 선순환 구조 구축
- 개발 서버 온프레미스 이전으로 월 약 \$1,000 AWS 리소스 비용 추가 절감 및 리소스 사용량 최적화
- 환경별로 역할을 분리해 장애를 격리하고 운영 안정성 확보

프로젝트 2: 온프레미스 CI/CD 파이프라인 구축 및 고도화

기여도 100% (인프라 및 파이프라인 전체 담당) 2024.12 ~ 현재 (운영 중)

문제

개발자가 수동으로 빌드 후 서버에 배포하는 방식으로, 배포 시 평균 30분이 소요되었으며 휴먼 에러로 인한 장애가 빈번하게 발생했습니다.

접근 전략

GitLab CI와 ArgoCD를 조합한 GitOps 기반 자동 배포 환경을 구축했습니다. 개발자가 Git Push만 하면 자동으로 빌드→테스트→배포가 진행되며, ArgoCD가 Kubernetes 클러스터 상태를 모니터링하여 선언적 배포를 수행합니다.

Kaniko로 Docker-in-Docker(privileged) 없이 Docker 이미지를 빌드하고, 멀티 환경(alpha/review/dev/staging) 배포를 단일 파이프라인에서 관리하도록 구성했습니다.

트러블슈팅

- 환경 구분 없이 `selfHeal=true` 를 적용하면 prod 변경이 사람 승인 없이 자동 반영되고 긴급 대응 유연성도 떨어지는 문제 → prod는 `selfHeal=false` + 수동 sync window로 배포를 게이트하고 dev/staging만 `selfHeal=true` 로 drift를 자동 보정, 모든 sync 이벤트를 Slack으로 알림 연동하여 변경 추적성과 안정성을 동시에 확보
- 데이터 업로드 경로를 키리스로 전환 — 기존 SSH scp/rsync 직결 방식을, S3 transit 버킷에 스테이징한 뒤 SSM SendCommand로 대상 인스턴스가 `aws s3 sync --exact-timestamps` 를 수행하는 방식으로 교체하여 SSH 키를 완전히 제거하고 배포 계정 키 유출 리스크를 차단

성과

- 배포 시간 67% 단축 (30분 → 10분), 데이터 배포 시 소스를 받아오는 단계를 `git fetch --depth=1 --filter=blob:none` 로 교체해 약 139초 → 약 30초로 단축
- GitOps 자동 배포를 환경별로 이원화 — dev/qa/review는 머지 기반 지속 배포(핵심 서비스 repo당 주 100회 내외)로 신규 피처의 Time to Market을 단축하고, prod는 수동 sync window 승인을 거쳐 주 5~7회만 릴리스해 안정성 확보
- 빌드·배포·설정 적용의 수동 단계를 파이프라인으로 대체해 휴먼 에러로 인한 배포 장애를 제거하고, 크로스레포 MasterData 파이프라인으로 데이터 변경 시 하위 서비스 자동 fan-out + 순환 트리거(loop) 원천 차단

프로젝트 3: 중앙 로깅·관측 플랫폼 (로그 수집 → 무결성·HA → 제품 분석)

기여도 100% (로그 수집·무결성·제품 분석까지 전 과정 단독 설계·구축) 2024.11 ~ 현재

로그가 분산된 온프레미스 환경에서 시작해 중앙 집중식 로깅을 3단계로 고도화(ELK → EFK → ECK Operator)하고, 수집기 데이터 무결성·HA를 강화한 뒤, 축적된 EFK 로그 파이프라인을 게임 유저 리텐션 분석 플랫폼으로 확장했습니다. build → operate → improve → analytics로 이어지는 관측 플랫폼 전 과정을 단독으로 설계·운영했습니다.

3-1. 중앙 집중식 로깅 시스템 구축 (ELK → EFK → ECK Operator 3단계 고도화, 2024.11 ~ 현재)

문제

온프레미스 환경에서 서버 및 컨테이너 로그가 분산되어 있어 장애 발생 시 원인 파악에 많은 시간이 소요되었으며, 통합 모니터링 체계도 없었습니다.

접근 전략

로깅 스택을 ELK → EFK → ECK Operator 3단계로 고도화했습니다. 초기 ELK로 중앙 집중식 로깅을 구축한 뒤, Logstash의 JVM 메모리 부담과 확장성 한계를 해소하기 위해 경량 Fluent Bit DaemonSet 수집 + Fluentd 가공 구조의 EFK로 전환했습니다.

이후 Elastic 공식 Helm 차트가 2년 이상 정체되자 CRD 기반 선언형 관리(ECK Operator)로 옮기며 Elastic Stack을 8.5.1 → 9.0.0으로 무중단 major 업그레이드했고, 온프레미스 EFK 스택을 AWS(EKS)용 `-aws` 스택으로 템플릿화해 같은 구조를 이식하면서 로그 수명주기(ILM)와 S3 스냅샷 보관을 IRSA로 정적 키 없이 자동화했습니다.

성과

- 로그 검색 시간 90% 단축 (개별 Pod 접속 확인 → Kibana 단일 인터페이스), MTTR 개선으로 서비스를 안정적으로 유지
- 무거운 Logstash를 경량 Fluent Bit + Fluentd 구조로 재설계하고, ECK Operator로 전환해 TLS 인증서·계정·업그레이드를 수동으로 관리하던 부담을 제거(CR spec.version 한 줄 변경으로 롤링 업그레이드), 2년간 정체된 Elastic Stack의 major 업그레이드를 무중단으로 완료 (8.5.1 → 9.0.0)
- 온프레미스 EFK 스택을 AWS(EKS)용 `-aws` 스택 5종으로 템플릿화해 같은 로깅 구조를 그대로 이식, 인덱스 수명주기(ILM)와 S3 스냅샷 보관을 IRSA web-identity로 정적 키 없이 자동화

3-2. Fluent Bit 로그 수집기 데이터 무결성 & HA 강화 (2026.03 ~ 2026.05)

Fluent Bit Pod 재시작 시 tail offset 소실로 인한 로그 재색인·누락 위험을 SQLite offset DB·전용 PVC로 해결 — 재색인 0건·중복 0건, 7개 alert로 미감지 장애 차단

3-3. 게임 유저 리텐션 분석 플랫폼 (EFK 로그 → 제품 분석, 2026.06 ~ 현재)

장애·검색용 EFK 스택을 Elasticsearch Transform(5분)으로 사용자당 1행 집계하는 제품 분석 플랫폼으로 확장 — Kibana 12패널로 D+1~D+30 리텐션·코호트 제공, 별도 SaaS 없이 구현

프로젝트 4: 개발 생산성 도구 구축 (APK 배포 봇 · 문서 자동화 · LLM 서비스 · Git 미러링 · 정적 파일 서버)

기여도 100% (전체 시스템 설계 및 개발) 2025.02 ~ 현재

개발팀의 반복적인 수동 작업을 자동화하고 생산성을 높이기 위해, 5가지 사내 도구를 설계·개발·운영했습니다.

4-1. Slack APK 배포 자동화 봇 (4주, 2025.02)

Jenkins APK 빌드 완료 시 Slack으로 완료 메시지·다운로드 QR을 자동 전송하는 Python 봇을 Kubernetes에 배포 — QA 전달 시간 90% 단축, 버전 혼동 제거

4-2. GitLab-Google Drive 문서 자동화 시스템 (2주, 2025.06)

GitLab Webhook·Google Drive API를 연동해 Push 시 마크다운을 Google Docs로 자동 변환·동기화하고 rclone `--checksum` 으로 최적화 — 문서 관리 시간 80% 단축(10분→2분), GitLab을 SSOT로 일원화

4-3. 사내 LLM 서비스 구축 (4주, 2025.11)

온프레미스 GPU 서버에 Ollama·Open WebUI를 Kubernetes로 배포하고 NFS 모델 저장소를 구성 — 개발팀 15명이 일 100건 쿼리 활용, 외부 SaaS 대체로 월 \$100 절감·데이터 유출 리스크 제거

4-4. Git 저장소 미러링 도구 개발 + Retry API 후속 강화 (4주 초기 개발, 2026.02 ~ 2026.06)

문제

CodeCommit·GitLab·GitHub 멀티 플랫폼 수동 동기화로 누락·지연이 발생해 내부 소스를 GitLab으로 통합해야 했고, mirror 실패 시 `kubectl exec` 로 webhook secret을 노출하는 수동 재시도밖에 없었습니다.

접근 전략

Go로 양방향 Git 미러링 도구 git-bridge를 개발해 변경 이벤트 기반 증분 동기화·루프 감지로 운영하고 Kubernetes에 배포했습니다.

이후 수동 재시도용 `POST /retry/mirror` 엔드포인트를 webhook secret과 분리된 전용 토큰 `RETRY_API_TOKEN` (constant-time 비교)으로 인증해 secret 노출 없이 복구하도록 했고, 비-fast-forward 회귀는 ref 패턴별로 방향을 고정하는 `ref_overrides` 와 `fail-open`(실패해도 막지 않고 통과) 원칙으로 차단했습니다.

성과

- 저장소 10개·일 평균 30건 동기화 100% 자동화로 수동 미러링을 제거하고 오픈소스 공개

- Retry API로 webhook secret 노출 없는 SRE 친화적 복구 경로를 확보하고 `ref_overrides` 방향 고정 가드로 비-fast-forward 회귀를 차단 — 테스트 전용 repo 6/6 PASS·프로덕션 격리

4-5. 정적 파일 서버 자체 개발 (오픈소스, 2026.03 ~ 현재)

문제

기존에 쓰던 [halverneus/static-file-server](#)는 단순 파일 서빙만 제공해 QA팀 APK 배포·사내 파일 공유에 필요한 운영 기능이 없었고, 업스트림 차트 유지보수도 사실상 멈춰 있었습니다.

접근 전략

Go 기반 경량 정적 파일 서버(static-file-server)를 직접 설계·개발해 운영에 필요한 기능을 자체 구현하고, GitHub Pages에 [Helm repository](#)를 운영해 배포 표준을 확립했습니다.

Kubernetes에는 자체 Helm chart로 배포하고 NFS 스토리지에 연결해 장기 보존 아티팩트를 안정적으로 호스팅하며, 기존 OSS 이미지 기반 배포를 걷어내고 halverneus/static-file-server를 완전히 대체했습니다.

성과

- Apache-2.0 오픈소스 공개, 일 평균 30건 다운로드·주 5회 APK 배포 트래픽 처리
- APK 배포·문서 공유·빌드 아티팩트 배포를 단일 도구로 통합하고, 업스트림 의존성을 제거해 보안 패치·기능 추가를 자체 개발 주기로 관리

프로젝트 5: Kubernetes 클러스터 운영 고도화

기여도 100% (단독 설계 — 모니터링·업그레이드 자동화·Helm 관리 프레임워크·NGF 마이그레이션·OSS Helm 차트 3종·스토리지 성능 최적화·Shell→Python 마이그레이션, 자동화 스크립트 4개로 표준 절차를 코드로 남겨 → 팀 재사용 가능) 2025.12 ~ 현재

클러스터를 더 잘 들여다보고 안정적으로 운영하기 위해 모니터링을 개편하고 K8s 업그레이드·Helm 관리를 자동화했으며, 네트워크 컨트롤러를 Gateway API 기반으로 마이그레이션했습니다.

NGF 마이그레이션과 ECK Operator 도입의 산출물을 OSS Helm 차트로 공개하고, control-plane / DB 노드의 SSD 마이그레이션 및 빌드 I/O 격리로 스토리지 성능 병목을 해소했으며, 인프라 배포/업그레이드 파이프라인을 단계별(wave) 순차 자동화 + Shell → Python 마이그레이션으로 정비했습니다.

5-1. 모니터링 & 알림 고도화 (2025.12 ~ 현재)

kube-prometheus-stack 기반 커스텀 알림·Grafana 대시보드를 설계하고 DB·node-exporter·Cilium 메트릭을 통합 — 9개 컴포넌트 그룹에 27개 커스텀 알림 운영, severity 라우팅·inhibit로 중복 제거

5-2. K8s 업그레이드 자동화 & Helm 차트 관리 프레임워크 (2025.12 ~ 현재)

Kubespary 업그레이드 자동화 스크립트와 다단계 헬스체크(노드·etcd·API Server)를 구축하고 Helm 프레임워크·4종 캐노니컬 템플릿 동기화로 버전 관리를 표준화 — 검증 시간 90% 단축, 인증서 자동 갱신으로 만료 장애 제거

5-3. Ingress-nginx → NGINX Gateway Fabric(NGF) 마이그레이션 (2026.04)

온프레미스 ingress-nginx 다수 인스턴스를 NGF 2.x 단일 컨트롤플레인+클래스별 Gateway CR로 전환 — MetalLB IP 유지로 DNS·방화벽은 그대로 두고 사고 없이 cutover(클래스당 30~60초), wildcard 인증서로 수동 갱신 50% 감소

5-4. OSS Helm 차트 3종 공개 — nginx-gateway-cr · elasticsearch-eck · kibana-eck (2026.04 ~ 2026.05)

NGF·ECK 산출물을 범용 차트로 다듬어 3종(nginx-gateway-cr·elasticsearch-eck·kibana-eck)을 Apache-2.0으로 ArtifactHub에 공개 — mgmt 클러스터를 cluster_uuid를 그대로 유지한 채 공용 OCI 차트로 무중단 전환

5-5. 스토리지 성능 최적화 — etcd / DB SSD 마이그레이션 + 빌드 I/O 격리 (2주, 2026.04 ~ 2026.05)

문제

온프레미스 control-plane VM과 게임 DB 워커 노드가 HDD에서 동작하며 GitLab Runner buildx 작업과 디스크 I/O 경합이 발생해, etcd WAL fsync p99가 약 2초, apiserver GET p99가 약 7초, 게임 DB COMMIT latency가 9~14초로 폭증하는 플레이 불가 장애가 반복됐습니다.

분석 결과 GitLab Runner의 대량 write 트래픽이 game-DB worker VM 디스크의 fsync queue를 점유하는 것이 근본 원인이었습니다.

접근 전략

MySQL fsync 파라미터 튜닝으로 디스크 fsync 부하를 즉시 완화한 뒤, control-plane VM을 SSD로 이전하며 **예상 15분 대비 다운타임 8.5분**으로 전환하고 etcd 헬스체크·Prometheus 지표로 검증했습니다. game-DB worker VM의 MySQL·Redis도 백업 → SSD 이전 → 복구 순서로 데이터 손실 없이 옮겼습니다.

마지막으로 별도 물리 서버에 빌드 전용 VM을 신설하고 taint/toleration으로 GitLab Runner buildx 작업을 격리해 I/O 경합을 원천 차단했으며, 자동화 스크립트로 향후 워커 노드도 동일 절차로 생성할 수 있게 했습니다.

성과

- **GitLab Runner buildx의 디스크 I/O 경합을 근본 원인으로 진단**하고 control-plane VM을 SSD로 이전해, 초 단위까지 치솟던 etcd fsync-apiserver 응답 지연을 정상 범위로 회복 — Kubernetes API 가용성 복구
- **게임 DB COMMIT 지연 9~14초 → ms 수준으로 회복**해 빌드 작업 중 게임 플레이 불가 장애 0건 달성, 빌드 전용 VM 신설로 buildx 워크로드를 DB 워크로드에서 물리적으로 격리해 I/O 경합 재발 원천 차단
- 자동화 스크립트 4개와 설정 파일로 절차를 코드화해, 향후 워커 노드 추가 시 동일 표준 절차 재사용 가능 (운영 표준화)

5-6. 인프라 배포/업그레이드 자동화 + Shell → Python 마이그레이션 (2026.03 ~ 현재)

문제

사내 Kubernetes 인프라 monorepo의 25개 컴포넌트가 helmfile + shell 약 39,000줄로 관리되어 신규 배포·차트 업그레이드·template 동기화가 전부 수동이었고, shell + awk는 멀티 환경 분기·복잡한 YAML 패칭·테스트에서 한계를 드러냈으며 CI에서 helmfile-plugin 설치만으로 5분이 걸렸습니다.

접근 전략

CI 파이프라인을 6단계 wave(bootstrap → core → security → db-redis → observability → apps)로 구성하고 MR diff를 감지해 변경 컴포넌트만 helmfile diff/apply하며, manual gate로 prod-safe 순차 배포를 표준화했습니다. CI가 25개 컴포넌트 차트의 신버전을 감지해 변경 chart-values diff·breaking change 노트를 렌더링한 자동 MR을 생성하도록 업그레이드도 자동화했습니다.

기존 shell + awk 자동화는 Python(stdlib only)으로 마이그레이션해 canonical template·동기화/백업 스크립트·CI 스크립트를 모듈화하고 컴포넌트별 업그레이드 모듈과 758개 단위 테스트를 정비했으며, 프로젝트 7의 helmfile-tools 이미지를 적용해 CI 준비 시간을 5분 → 약 10초로 단축했습니다.

성과

- **25개 컴포넌트 단계별(wave) 순차 자동 배포 + 업그레이드 MR 자동 생성 구축**으로 인프라 배포/업그레이드 운영 부담 대폭 절감
- **Shell + awk 자동화를 Python(stdlib only)으로 마이그레이션** — 멀티 환경 분기·복잡한 YAML 패칭의 한계를 걷어내고 가독성·테스트 가능성을 동시에 확보 (758 unittest 통과)
- helmfile-tools image 적용으로 **CI 환경 준비 시간 5분 → 약 10초 (약 95% 단축)**, 로컬(macOS venv) + 원격(CI container) 동일 코드 보장으로 개발 → 배포 사이의 환경 차이 문제 제거

5-7. helmfile Push → ArgoCD Pull-GitOps 전환 (App-of-apps 컨트롤플레인, 2026.06 ~ 현재)

문제

플랫폼 릴리스를 helmfile push 방식으로 배포하다 보니 배포 주체가 CI 러너에 묶여 있었고, 멀티클러스터로 확장하면서 클러스터별 상태 드리프트를 선언적으로 바로잡을 pull 기반 GitOps 컨트롤플레인이 필요했습니다.

접근 전략

helmfile push 배포를 ArgoCD pull-GitOps로 전환했습니다. Matrix + Git files 제너레이터 기반 ApplicationSet(여러 Application을 자동으로 찍어내는 컨트롤러)이 각 컴포넌트의 `argocd/*.yaml` 마커 파일을 읽어 Application을 자동 생성하도록 구성해, 온프레미스 16개 + AWS 8개 총 24개 플랫폼 릴리스를 2개 클러스터에 등록했습니다.

기존 helm 릴리스는 재생성 없이 그대로 ArgoCD 관리로 넘겨받아(무중단 adoption) 배포 순서를 보존했고, App-of-apps 계층에 연쇄 삭제 방지(prune-cascade) 안전장치와 AppProject 최소 권한을 적용해 실수로 인한 대량 삭제 리스크와 장애 영향 범위(blast-radius)를 축소했습니다.

성과

- helmfile push → ArgoCD pull-GitOps 전환으로 2개 클러스터 24개 플랫폼 릴리스(온프레미스 16 + AWS 8)를 단일 컨트롤플레인에서 선언 관리, 기존 helm 릴리스를 재생성 없이(무중단) adopt해 배포 순서 보존
- App-of-apps 연쇄 삭제 방지(prune-cascade) 안전장치로 GitOps 대량 삭제 리스크 차단, AppProject 최소 권한 whitelist로 장애 영향 범위(blast-radius) 축소

프로젝트 6: 인프라 보안 체계 구축

기여도 100% (설계, 배포, SSO 연동) 2026.04 ~ 현재

인프라 운영에 필요한 시크릿 관리, 중앙 인증, 원격 접속을 순차적으로 구축하고 있습니다.

6-1. Vaultwarden 비밀번호 관리 시스템 (1주, 2026.04 ~ 현재)

Slack·이메일·개인 저장소에 흩어져 있던 사내 시크릿을 Vaultwarden으로 중앙화 — Kubernetes에 Helm으로 배포하고 GitLab SSO(OIDC)·RBAC 연동, 사용자 70명·시크릿 50건 전환, SaaS 비용 0원

6-2. Keycloak Operator 기반 중앙 인증(SSO) 시스템 도입 (2026.04 ~ 현재)

문제

ArgoCD·Harbor·Vaultwarden 등 사내 인프라 도구가 각각 GitLab을 OIDC IdP(로그인 인증을 제공하는 신원 제공자)로 직접 가져다 쓰다 보니, GitLab 자체의 사용자·세션·MFA(다중 인증) 정책에 대한 의존이 계속 깊어졌습니다.

앞으로 LDAP·MFA·세션 정책을 한곳에서 관리하려면, 로그인을 중개하는 별도의 중앙 인증 서버(broker, Keycloak)가 필요한 시점이었습니다.

접근 전략

Keycloak Operator + KeycloakCR + KeycloakRealmImport를 PostgreSQL 백엔드와 함께 도입하고, Keycloak이 GitLab 로그인을 중개(brokering)하도록 구성해 사용자는 기존 GitLab 계정 그대로 로그인합니다. Realm / Client / IdP / Group / Mapper 설정을 부트스트랩 스크립트로 코드화(codify)하고, Harbor·ArgoCD·Vaultwarden이 GitLab에 직접 연동하던 OIDC를 Keycloak을 거치도록 전환했습니다.

권한(RBAC)은 GitLab group을 Keycloak group mapper로 중개한 뒤 ArgoCD role mapping에 연결해 권한 운영의 단일 기준을 Keycloak으로 통일했습니다.

성과

- **ArgoCD / Harbor / Vaultwarden OIDC 통합 완료, 무중단, 검증 PASS** — GitLab 직접 연동 의존성 해소
- GitLab group(`server` , `global-admin`) → Keycloak group mapper → ArgoCD role로 이어지는 권한 흐름을 하나로 확립해, 누가 무엇을 할 수 있는지를 Keycloak에서만 정하도록 일원화, 향후 LDAP 연동(federation)·MFA 강제·세션 정책 중앙화의 기반 마련
- Keycloak Operator 도입 과정에서 직접 부딪힌 5가지 시행착오(IdP providerId / scope / mapper config / Operator idempotency / Harbor user mapping)를 설계 문서에 정리해 향후 클러스터 재구축 시 재발 방지

6-3. GitOps 시크릿 관리 — External Secrets Operator · Sealed Secrets (2026.05 ~ 현재)

DB 비밀번호를 Helm values 평문으로 Git에 커밋하던 구조를 걷어내고, External Secrets Operator로 AWS Secrets Manager를 연결해 ExternalSecret 참조로 전환, Harbor pull secret은 cluster-wide SealedSecret으로 표준화 — AWS SM을 시크릿의 단일 관리 기준으로 일원화

6-4. OpenVPN 원격 접속 서버 (2026.07)

iptime 공유기 내장 VPN의 동시 접속 5개 제한을 해소하기 위해 물리 서버에 OpenVPN 원격 접속 서버를 자체 구축 — easy-rsa PKI 기반 클라이언트별 EC 인증서(prime256v1) 인증 + tls-crypt 제어 채널 암호화·인증, AES-256-GCM·TLS 1.2+, full-tunnel을 기본으로 하되 split-tunnel 전환도 지원하고, 터널 대역 10.8.0.0/24를 SNAT(MASQUERADE)로 사내 물리 서버망에 라우팅(동시 접속 최대 50 설정).

먹통 bash 설치 스크립트(PKI 빌드 → server.conf 렌더 → NAT systemd 유닛 → 서비스 기동)와 클라이언트 인증서 발급·폐기(revoke + CRL 갱신) 라이프사이클 스크립트로 운영을 자동화하고 관리자 Mac에서 SSH로 원격 제어

프로젝트 7: 공용 Toolchain Image 계층 구조 + 자동 버전 사이클

기여도 100% (아키텍처 설계 · CI 표준화 · 자동화 파이프라인 개발) 2025.09 ~ 2026.05

문제

GitLab CI 표준화 범위 안의 15개 repo(1개 template provider + 14개 consumer)에서 각 잡이 alpine 베이스에 helm·helmfile·kubectl·crane·aws-cli 등을 매번 설치하면서 첫 번째 잡 준비 시간이 평균 5분에 달했고, 도구 버전이 14개 consumer repo에 흩어져 있어 신버전 추가 시 일괄 갱신에 모든 consumer repo를 함께 손대야 했습니다. `:latest` 태그 의존으로 재현성이 떨어지고 취약점 추적도 어려웠습니다.

접근 전략

Layer 1 (Mirror): 외부 레지스트리(Docker Hub, GHCR)의 도구 이미지를 `crane copy` 로 사내 Harbor에 multi-arch manifest(여러 CPU 아키텍처를 묶은 이미지 목록)를 보존한 채 미러링.

Layer 2 (Custom): Layer 1 위에 도구를 미리 설치해 둔 대용량 이미지 7종(helmfile-tools, node-build, go-build, rclone-backup, aws-cli-tools, kaniko-build, terraform-tools)을 빌드해 consumer repo가 image pin만으로 도구를 즉시 사용할 수 있도록 표준화.

Tier 1~2 SSOT(단일 관리 기준): `.toolchain_build_custom` 공용 template + `TOOLCHAIN_*_TAG` 중앙 변수를 도입해 consumer repo가 image tag를 직접 추적하지 않고 중앙 변수만 바라보도록 분리.

공용 CI 템플릿 SSOT: repo마다 복사해 쓰던 CI 잡을 중앙 공용 CI 템플릿 repo로 추출해 각 repo가 `include` 로 참조하도록 전환. auth(키리스 AWS 인증) / build(kaniko) / deploy(ArgoCD) / backup(Google Drive) 잡 템플릿과 공용 앵커(git-ssh-rules·job-config·packages)를 단일 소스로 관리해 복사본마다 설정이 갈라지는 문제를 제거.

Phase 4 자동 사이클: cron 트리거로 다음 4단계가 자동 진행되어 사람이 직접 누르는 클릭 2회(pipeline trigger + Tier 2 MR merge)만으로 연쇄 전파 완성:

- (1) upstream 신버전 감지
- (2) Layer 1 mirror auto-MR
- (3) Layer 2 rebuild + Tier 2 TOOLCHAIN_*_TAG 동기화 auto-MR
- (4) 14개 consumer repo(template provider repo 제외)에 자동 전파

성과

- **CI 첫 번째 잡 준비 시간 5분 → 약 10초 (약 95% 단축)**, 14개 consumer repo에 도구 버전 일관성 100% 확보(총 15개 repo), 버전 거버넌스 4-tier(ARG → 중앙 변수 → consumer 분리 → lint drift 감지)로 12곳 이상에 흩어져 있던 버전 지정 지점을 한 흐름으로 정리
- Phase 4 cron 자동 사이클로 helm·helmfile·kubectl·crane·rclone 등 5종 이상 도구의 신버전 연쇄 전파를 사람 클릭 2회로 완료하고, minor-guard 정책(helm/helmfile/kubectl)으로 호환성이 깨지는 상황을 사전 차단
- 25개 컴포넌트를 리스크 티어(T1 주간 / T2 격주 / T3 월간 + pre-flight)로 분류한 자동 업그레이드 파이프라인 구축(신버전 감지 → 자동 MR → Slack 알림 → wave 순차 적용, MAJOR_PIN 가드·T3 atomic 롤백), docker buildx로 amd64 + arm64 멀티아치를 빌드하고 외부 이미지는 crane으로

프로젝트 8: AWS EKS 프로덕션 게임 런칭 플랫폼 구축 (신규 외부 퍼블리싱 게임)

기여도 100% (그린필드 EKS 단독 설계·구축) 2026.06 ~ 현재 (구축·운영 중)

문제

라이브 게임 환경(AWS QA·Prod)과 별개로, 신규 외부 퍼블리싱 게임은 온프레미스 개발 클러스터 하나로만 운영 중이었고, 프로덕션 런칭을 위해 확장성·가용성을 갖춘 별도 AWS EKS 프로덕션 환경이 필요했습니다. 온프레미스만으로는 글로벌 트래픽 대응과 오토스케일링에 한계가 있었습니다.

접근 전략

eu-central-1 리전에 그린필드 EKS 클러스터를 구축해 온프레미스 개발 + AWS 프로덕션의 멀티클러스터 하이브리드로 확장하고, 플랫폼 컴포넌트 약 11종을 전부 GitOps(ArgoCD)로 선언 관리했습니다.

오토스케일링·네트워킹: Cluster Autoscaler 대신 Karpenter를 채택해(다수 스팟 family를 단일 프로비저너로 통합·빠른 노드 기동), 스팟 중단에 강한 다중 family 스팟 오토스케일링과 게임 서버 전용 on-demand NodePool을 구성하고, AWS Load Balancer Controller(ALB/NLB) + ExternalDNS(Route53) + wildcard ACM으로 인그레스·DNS·TLS를 자동화했습니다.

로깅·시크릿: eck-operator 기반 EFK 스택(Elasticsearch 3노드 3-AZ, EBS gp3, S3 스냅샷 SLM)을 HA로 구성하고, External Secrets Operator로 시크릿을 외부화했습니다. 모든 ServiceAccount 인증은 IRSA / Pod Identity로 처리해 정적 키를 제거했습니다.

인증·배포: ArgoCD에 GitLab OAuth SSO + 게임 스코프 RBAC를 연동하고, 클라이언트 아티팩트는 Jenkins → S3 + CloudFront로, 서버 매니페스트는 S3 transit 버킷 + SSM SendCommand(SSH 없이 원격 명령 실행)로 bastion의 EFS 마운트에 전달했습니다. bastion을 Name 태그로 동적 탐색해 SSH 키 없이 배포했습니다.

트러블슈팅

- 게임 프로덕션 Pod가 실사용량의 약 4배에 달하는 메모리를 예약해 노드가 불필요하게 증설되는 node sprawl이 발생 → 실측 프로파일 기반으로 memory requests를 right-sizing해 노드 증설을 정상화하고 스팟 비용 효율 확보
- 롤링 배포 중 ALB가 아직 준비되지 않은 Pod로 트래픽을 전달하는 문제 → ALB Pod readiness gate + PodDisruptionBudget + graceful drain(연결을 안전하게 종료)을 조합해 무중단 배포 확보

성과

- 온프레미스 개발 클러스터 하나 → 온프레미스 + AWS 프로덕션 멀티클러스터 하이브리드로 확장, 신규 외부 퍼블리싱 게임의 클라우드 프로덕션 런칭 기반 확보
- 약 11종 플랫폼 컴포넌트를 100% GitOps(ArgoCD)로 선언 관리하고, IRSA / Pod Identity + SSM SendCommand로 정적 키 한 장 없이 인증·배포하도록 정리해 bastion SSH 키 유출 리스크 제거
- ArgoCD 앱 마커와 프로덕션 CI/CD 전환 문서로, 단독 구축이지만 팀이 동일 절차로 운영·재구축할 수 있도록 인수인계까지 문서로 남김

프로젝트 9: Observability 고도화 — 분산 추적 & Progressive Delivery

기여도 100% (단독 설계·구축 — OTel 추적 파이프라인·Argo Rollouts Canary·소프트 런칭 운영) 2026.06 ~ 현재

Project 8의 AWS EKS 프로덕션 게임 런칭 플랫폼 위에, 신규 외부 퍼블리싱 게임의 소프트 런칭에 대비해 관측과 배포를 보강했습니다.

OpenTelemetry 기반 엔드투엔드 분산 추적 파이프라인으로 로깅·메트릭만 있던 기존 관측 플랫폼에 '추적' 축을 채우고, Argo Rollouts로 무중단 Canary 배포를 도입했으며, 이 스택 위에서 소프트 런칭 실패트래픽을 안정적으로 받아냈습니다.

9-1. OpenTelemetry 엔드투엔드 분산 추적 파이프라인 (OTel + Tempo)

문제

기존 중앙 관측 플랫폼은 로깅(EFK)과 메트릭(kube-prometheus-stack)만 갖추고 '추적(trace)' 축이 비어 있어, 요청이 서비스 사이를 어떻게 흐르는지, 어디서 지연이 발생하는지 확인할 수 없었습니다. 프로덕션 런칭을 앞두고 서비스 간 지연·에러의 원인을 분석할 수단이 필요했습니다.

접근 전략

AWS EKS에 OpenTelemetry Collector + Grafana Tempo(S3 저장, EKS Pod Identity로 정적 키 없음) + OpenTelemetry Operator(auto-instrumentation으로 앱 이미지 변경 없이 언어 SDK 주입)로 분산 추적 스택을 구축했습니다. 여기서 핵심은 샘플링 지점을 나눈 것입니다.

RED 메트릭은 샘플링 이전 100% span에서 생성해 정확도를 확보하고, trace 저장 단계에만 tail sampling(모든 에러 + 500ms 초과 + 나머지 10% 보존)을 적용해 추적 저장 비용을 절감했습니다. Grafana에서 Metrics ↔ Traces ↔ Logs 3-signal을 서로 오갈 수 있게 연결해, 로깅·메트릭만 있던 기존 관측 플랫폼(Project 3)에 비어 있던 '추적' 축을 채웠습니다.

성과

- 로깅·메트릭만 있던 관측 플랫폼에 '추적' 축을 더해 Metrics ↔ Traces ↔ Logs 3-signal을 서로 오갈 수 있게 연결
- RED 메트릭 생성(샘플링 이전 100% span)과 trace 저장(tail sampling)을 분리해 정확도는 지키고 저장 비용은 절감
- auto-instrumentation으로 앱 이미지 변경 없이 언어 SDK를 주입하고, EKS Pod Identity로 정적 키 없이 S3에 접근

9-2. Argo Rollouts 기반 Progressive Delivery (Canary)

문제

프로덕션 게임 서비스는 배포 중에도 라이브 트래픽을 끊지 않아야 했고, 문제가 생기면 즉시 롤백할 수단이 필요했습니다. 롤링 업데이트만으로는 old/new가 안전하게 공존하며 넘어가도록 전환 과정을 통제하기 어려웠습니다.

접근 전략

기존 Blue-Green은 승격 시 ALB target group을 한 번에 교체해, 순간 healthy target이 0이 되고 그 틈에 ALB가 503을 반환하는 경향이 있었습니다 (2026-07-14 관측). 그래서 Argo Rollouts Canary로 전환했습니다.

같은 ALB target group에서 `maxUnavailable: 0` + add-before-remove(새 Pod를 먼저 등록한 뒤 구 Pod를 제거)로 old/new가 순간 공존하며 넘어가 503 레이스를 제거했고, 게임 서비스가 stateless HTTP(JWT + Redis/TypeORM로 상태 외부화)라 전환 중 old/new에 트래픽이 섞여 들어와도 안전합니다. 공용 base-aws Helm 차트에서 배포 전략을 opt-in으로 켜고 끄도록 템플릿화해, 프로덕션 게임 메인 서비스 1개만 Canary로 전환하고(공개 트래픽이 없는 형제 내부 서비스는 일반 Deployment 유지), 컨트롤러는 leader-election HA + PDB로 구성해 노드 drain·Karpenter consolidation 중에도 롤아웃이 멈추지 않습니다.

성과

- 프로덕션 게임 서비스를 Blue-Green에서 Canary로 전환해, 승격 시 ALB의 healthy target이 순간 0이 되며 발생하던 503 레이스를 제거하고 같은 target group에서 maxUnavailable:0·add-before-remove로 무중단 전환 확보
- 컨트롤러를 leader-election HA + PDB로 구성해 노드 drain·Karpenter consolidation 중에도 롤아웃이 멈추지 않도록 유지

9-3. 소프트 런칭 운영 (2026.07 ~ 현재)

성과

- 2026.07.14(KST) 소프트 런칭 개시 — 앞의 스택 위에서 신규 외부 퍼블리싱 게임을 제한 공개로 열어 그 소규모 실트래픽을 무중단 Canary 배포와 3-signal 관측(실 로그에서 trace_id/span_id 상관관계 검증)으로 안정적으로 받아냄
- HPA·Karpenter 오토스케일링은 2026 Q4 글로벌 정식 런칭을 대비해 프로비저닝만 완료(소프트 런칭 트래픽에서는 스케일업 미발생)
- 2026 Q4(9~10월) 글로벌 정식 런칭에 필요한 관측·배포·오토스케일링을 모두 갖추

너디스타

2023.03 ~ 2024.07

DevOps Engineer

게임·블록체인 기반 스타트업에서 DevOps 엔지니어로 근무하며, AWS에서 GCP로 대규모 클라우드 마이그레이션을 총괄했습니다.

GCP Startup Program 기반 비용 최적화와 글로벌 네트워크 성능을 목표로, 게임 점검 시간을 활용해 최소 다운타임으로 전체 서비스를 이전했습니다. 소스 저장소는 온프레미스를 GitLab, Production을 GitHub로 이원화해 관리했습니다.

프로젝트 1: AWS → GCP 대규모 클라우드 마이그레이션 및 CI/CD 구축

기여도: 인프라 아키텍처 설계·마이그레이션 전략 수립 및 실행 총괄 70% (나머지 30%는 서버 개발팀의 application 측 마이그레이션 협업), CI/CD 파이프라인 구축 100% 10개월 (2023.03 ~ 2023.12)

문제

AWS 비용이 지속적으로 상승하고 있었으며(월 \$10,000+), 특히 NAT Gateway와 데이터 전송 비용이 높았습니다. 마침 GCP Startup Program 참여 기회가 생겨, 비용 절감과 멀티 리전 게임 서비스를 위한 글로벌 로드밸런싱 확보를 목표로 GCP 마이그레이션을 추진했습니다.

접근 전략

3단계 마이그레이션 전략을 수립해 단계별로 실행했습니다 — 1단계 개발 환경을 GCP에 먼저 구축·검증, 2단계 QA 환경 이전, 3단계 프로덕션 이전.

네트워크는 Shared VPC(Host Project / Service Project 분리)로 설계하고, GitHub Actions의 GCP 인증은 Workload Identity Federation으로 전환해, 서비스 계정 키 발급·순환을 OIDC 단명 토큰으로 대체하고 키 관리 부담을 없앴습니다. DNS는 서브도메인 위임(Hosting.kr → AWS Route53 → GCP Cloud DNS)으로 사전 검증한 뒤, 최종 NS 변경 시점에만 점검을 공지하고 cutover 했습니다.

비용·보안 측면에서는 Filestore의 SSD 최소 2.5TB 문제를 Compute Engine + pd-balanced 1TB 디스크 기반 NFS 서버 자체 구축으로 우회하고, Cloud Armor로 리전 차단 + IP 화이트리스트 WAF(웹 방화벽) 정책을 구성했습니다.

Terraform으로 GCP 인프라를 IaC화(IAM 제외 100%)하고, 마이그레이션 완료 후 GitLab CI와 ArgoCD를 조합한 GitOps CI/CD 파이프라인을 구축해 Helm Chart로 환경별 설정을 표준화하고, 모든 배포를 Git 커밋으로 되짚을 수 있게 했습니다.

트러블슈팅

- AWS ALB 기반 라우팅을 GCP 글로벌 로드밸런서로 전환하는 과정에서, AWS ALB는 헬스체크에 실패한 백엔드에도 트래픽을 통과시키지만 GCP LB는 이를 차단하는 차이 때문에 트래픽 라우팅 문제 발생.
GCP NEG(Network Endpoint Group) 구조를 분석하여 GKE 워크로드에 맞는 헬스체크·백엔드 설정으로 재구성
- GKE Ingress + BackendConfig의 healthcheck requestPath 응답이 누락되어 503 에러 발생. 해당 경로 응답을 보장하고 ManagedCertificate / FrontendConfig 설정과 서로 맞춰 외부 진입 트래픽 정상화
- Cloud CDN 마이그레이션 후 HTTP → HTTPS 리다이렉트 누락으로 게임 클라이언트 파일 다운로드가 실패.
URL Map을 HTTP / HTTPS 두 개로 분리(`default_url_redirect.https_redirect=true`) 하고 HTTP forwarding rule에 80 포트 target HTTP proxy를 별도 매핑하여 해결

- DNS를 GCP Cloud DNS로 위임한 뒤 Google Search Console에 도메인이 자동 등록되지 않아 검색 노출·도메인 소유 확인에 문제 발생. Search Console이 발급한 verification TXT 레코드를 Cloud DNS에 추가해 도메인 소유권을 검증한 뒤 재등록하여 해결

성과

- 클라우드 비용 50% 절감 (월 \$10,000 → \$5,000), 연간 약 \$60,000 비용 절약
- 3단계 전략으로 전체 서비스 마이그레이션 완료. 리소스 복사 방식으로 다운타임 최소화하고, 최종 DNS 전환 시에는 게임 서비스 특성을 반영해 2시간 내외 점검 공지 후 전환
- 전체 GCP 인프라를 Terraform 코드로 관리하도록 IaC화 (TFLint/Checkov 정적 분석 및 Terratest 단위 테스트 도입으로 코드 품질 확보)
- 배포 시간 50% 단축 (40분 → 20분), 롤백 시간 95% 감소 (30분 → 1~2분)
- 주간 배포 횟수 3배 증가, Git 커밋 기반 배포 이력 100% 추적 가능
- Workload Identity Federation 도입으로 GitHub Actions의 GCP 서비스 계정 키 관리 부담을 제거하고 키 유출 리스크를 원천 차단, 단명 토큰 기반 보안 모델로 전환

프로젝트 2: 게임 데이터 수집 자동화 시스템 개발

기여도 100% (Python 스크립트 개발 및 운영) 3개월 (2024.01 ~ 2024.03)

문제

게임 및 블록체인 데이터를 수동으로 수집하고 있어 분석가가 매일 2~3시간을 데이터 수집에 소비했으며, 휴먼 에러로 인한 데이터 누락이 빈번하게 발생했습니다.

접근 전략

MongoDB · CloudSQL · Google Analytics · Dune 등 멀티 데이터 소스를 BigQuery로 적재하고, 분석가용 Google Sheets까지 자동 갱신되는 데이터 파이프라인을 GCP 서비스 조합으로 구축했습니다. MongoDB는 Dataflow Flex Template 기반 ETL로 BigQuery에 적재하고, CloudSQL은 BigQuery의 직접 connection 기능을 활용하며, GA / Dune은 각자의 API (Dune은 별도 API Key)로 호출하여 일관된 인덱스 스키마로 정규화했습니다. Python Cloud Function이 BigQuery 쿼리 결과를 Google Sheets API로 적재하고, Cloud Scheduler가 Daily (전일 데이터) / Monthly (전월 데이터) 두 가지 주기로 자동 트리거합니다. BigQuery 적재 후에는 별도 중복 제거 Cloud Function이 정기 실행되어 데이터 정합성을 유지합니다. 전체 인프라 (BigQuery Dataset · Cloud Function · Scheduler · Cloud Storage · Artifact Registry · IAM 권한 · Service Account)는 Terraform으로 IaC 관리하여 환경 재구축과 변경 추적을 표준화했습니다.

트러블슈팅

- CloudSQL → BigQuery 직접 connection(federated query)이 리전 불일치와 connection Service Account 권한 부족으로 실패. connection을 동일 리전에 생성하고 connection SA에 Cloud SQL Client 권한을 부여하여 해결
- Dune API의 execute → poll → fetch 비동기 실행 구조와 rate limit으로 결과 누락·타임아웃 발생. 실행 상태 폴링 + 백오프로 결과 수신을 보장하고 API Key를 Secret Manager로 분리
- Daily 재실행 시 동일 행이 BigQuery에 재적재되는 비역등 문제 발생. 중복 제거 Cloud Function에 MERGE / ROW_NUMBER() 정합성 로직을 적용해 적재를 멍등화(재실행해도 결과 동일)하고 누락·중복 0 유지
- BigQuery 결과를 Google Sheets API로 적재할 때 단일 시트 1천만 셀 한도·write 쿼터와 분당 요청 한도(429)에 걸려 갱신이 실패. values.batchUpdate 체크 분할과 시트 분할로 셀 한도를 우회하고, 호출 pacing(throttle)·배치 처리·exponential backoff로 쿼터 내에서 동작하도록 구현

성과

- 데이터 수집 자동화 95% 달성 (일 2~3시간 수동 작업 제거)
- 자동화로 휴먼 에러 제거, 데이터 누락률 0% 달성
- 전체 인프라 Terraform IaC 화로 동일 파이프라인을 다른 GCP 프로젝트에 30분 내 재구축 가능, 변경 이력 100% Git 추적

프로젝트 3: PLG Stack 기반 모니터링 시스템 구축

기여도 100% (시스템 설계 및 구축) 4개월 (2024.04 ~ 2024.07)

문제

Kubernetes 클러스터와 애플리케이션의 실시간 모니터링 체계가 없어 장애 발생 시 사후 대응만 가능했으며, 원인 파악에 과도한 시간이 소요되었습니다.

접근 전략

Prometheus로 메트릭을, Loki로 로그를 수집하여 Grafana 대시보드로 통합 시각화하는 PLG Stack을 구축했습니다. CPU·메모리·네트워크 사용률과 애플리케이션 로그를 실시간으로 모니터링하고, 임계치 초과 시 Slack 알림을 자동 발송하도록 구성했습니다.

성과

- 임계치 기반 알림으로 장애 사전 감지 체계 확립
- 실시간 모니터링 및 알림 체계로 장애 대응 시간 단축
- 서비스 가용성 향상
- SLI/SLO 수립과 Alertmanager 알림 노이즈 정리로 운영팀 피로도 감소, 실제 장애 감지 정확도 향상

PaaS(Kubernetes + OpenStack + Ceph) 기반 인프라 솔루션을 고객사에 구축하고 기술 지원을 제공했습니다. 금융권·공공기관 온프레미스 클라우드 인프라를 설계·구축하고, 고객사 운영팀 교육을 직접 맡았습니다.

프로젝트 1: 고객사 맞춤형 PaaS 솔루션 구축

기여도 100% (인프라 설계 및 Kubernetes 클러스터 구축) 11개월 (2022.04 ~ 2023.03)

문제

고객사마다 요구하는 환경과 서버 스펙이 상이하여, 표준화된 솔루션으로는 대응이 어려웠습니다.

접근 전략

고객사 요구사항에 맞춰 Kubernetes 클러스터를 HA(High Availability) 구성으로 설계하고, OpenStack으로 가상머신 관리 환경을 구축했습니다. Ceph로 블록·파일·오브젝트 스토리지를 통합 제공하고, 고객사 운영팀에 인프라 운영 교육을 병행했습니다.

성과

- 5개 고객사에 PaaS 솔루션 성공적으로 구축 및 납품
- Kubernetes 클러스터 HA 구성으로 안정적 운영
- Ansible 기반 서버 프로비저닝·구성 관리 자동화로 고객사별 배포 시간 단축, 환경 불일치(Configuration Drift) 0건 달성

프로젝트 2: DP사 Kubernetes 운영 교육 프로그램 설계·운영

기여도 100% (교육 설계 및 진행) 6개월 (2022.06 ~ 2022.11)

문제

DP사 운영팀이 Kubernetes·컨테이너 기반 인프라 운영 경험이 부족하여, PaaS 솔루션 도입 후 자체 운영에 어려움이 예상되었습니다.

접근 전략

Kubernetes 기초부터 클러스터 운영·트러블슈팅까지 단계별 교육 커리큘럼을 설계하고, 실습 환경을 구성해 실무 중심으로 운영했습니다. OpenStack·Ceph 스토리지 운영 교육도 병행해 전체 PaaS 스택 운영 역량을 확보하도록 도왔습니다.

성과

- 수강자 평균 10명 · 회당 교육 4시간 규모로 커리큘럼을 운영하여 DP사 운영팀의 Kubernetes 자체 운영 체계 확립
- 교육 완료 후 고객사 기술 문의 건수 감소, 고객사가 장애를 스스로 대응할 수 있게 됨
- 교육 자료를 표준화하여 이후 타 고객사 교육에도 재활용

프로젝트 3: Kubernetes 인증서 자동 갱신 프로세스 개선

기여도 100% (단독 수행, 전체 고객사 배포로 확산) 2주 (2022.11)

문제

Kubernetes 클러스터 인증서가 1년마다 만료되어 고객사마다 연간 약 5시간의 갱신 작업이 필요했으며, 만료 시 서비스 장애가 발생할 위험이 있었습니다.

접근 전략

Kubernetes 인증서 관리 프로세스를 분석하고, kubeadm 설정을 수정하여 인증서 유효기간을 1년에서 10년으로 연장했습니다. 기존 운영 중인 클러스터에도 적용 가능한 자동화 스크립트를 개발하여 전체 고객사에 배포했습니다.

트러블슈팅

- kubeadm 설정 수정 방식은 kubeadm으로 초기화한 클러스터에만 적용 가능했고, v1.15.x / v1.16.x에서는 `kubeadm alpha certs renew`의 알려진 버그로 일부 인증서가 갱신되지 않았습니다.
이런 버전에서는 kubeadm 명령에 의존하지 않고, `/etc/kubernetes` 백업 후 인증서 파일을 직접 10년으로 재생성하는 공개 오픈소스 스크립트 (`update-kube-cert`)를 도입했습니다. etcd·apiserver·controller-manager·scheduler·kubelet 재시작 절차와 묶어 적용해 버전에 관계없이 동작하도록 했습니다.

성과

- 인증서 갱신 주기 10배 연장 (연 1회 → 10년에 1회)
- 고객사 운영 부담 연간 약 50시간 절감 (고객사 10곳 기준)
- 인증서 만료로 인한 장애 리스크 제거